

信号 2 (进程间关系)

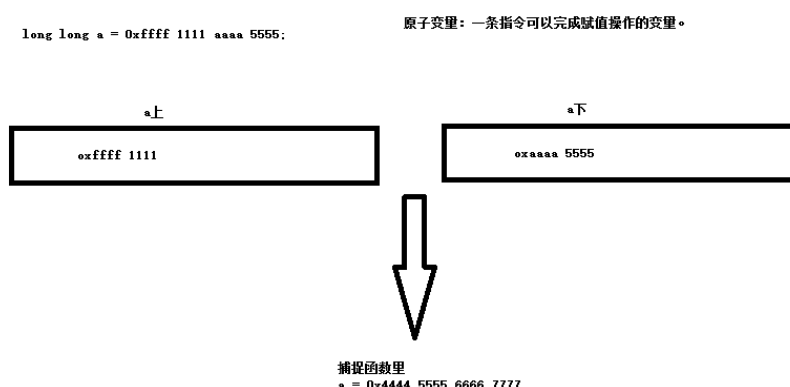
1、避免异步I/O的类型

平台下的原子类型: `sig_atomic_t`

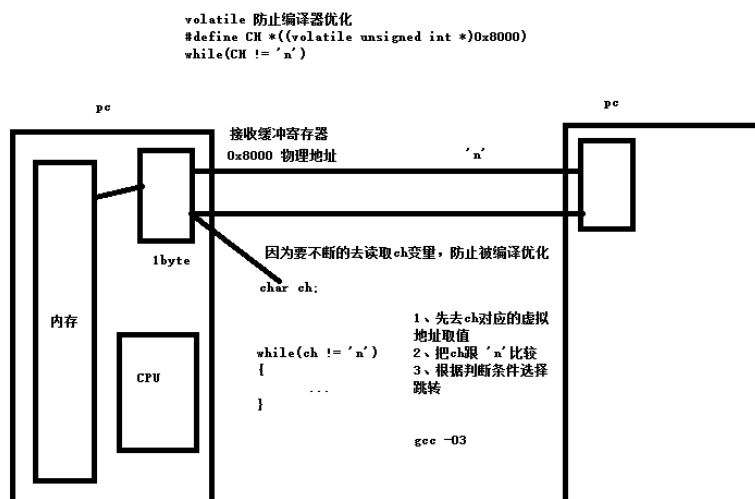
原子操作是在一个机器周期里完成一个动作, 不受CPU进程调度的影响。

捕捉函数尽量使用原子操作, 避免CPU进程调度干扰运行结果。

`long long a`; 变量a 在32位操作系统下就不能完成原子变量。`long long`类型占64位



2、volatile 防止编译器开启优化选项时, 优化对内存的读写操作



3、SIGCHLD的产生条件

- 1、子进程终止时；
- 2、子进程接受到SIGSTOP信号停止时；
- 3、子进程处在停止态，接受到SIGCONT后唤醒时；
- 4、 sigchild1.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <signal.h>
4
5
6 void do_sig_child(int num)
7 {
8     printf("child  %d is over\n",wait(NULL));
9 }
10
11
12 void sys_err(char *str)
13 {
14     perror(str);
15     exit(1);
16 }
17
18
19 int main(void)
20 {
21     pid_t pid;
22     if((pid = fork()) < 0)
23     {
24         sys_err("fork");
25     }
26     else if(pid == 0)
27     {
28         //子进程结束发送了SIGCHLD信号
29         int n = 5;
30         while(n--)
```

```

31     {
32         printf("child\n");
33         sleep(1);
34     }
35 }
36 else
37 {
38     //捕捉
39     struct sigaction act;
40     act.sa_handler = do_sig_child;
41     sigemptyset(&act.sa_mask);
42     act.sa_flags = 0;
43     sigaction(SIGCHLD,&act,NULL);
44
45
46     //父进程自己干自己的事
47     while(1)
48     {
49         printf("parent\n");
50         sleep(1);
51     }
52 }
53
54
55 return 0;
56 }

```

5、 sigchild2.c

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <signal.h>
4 #include <sys/wait.h>
5 #include <sys/types.h>
6
7
8 void do_sig_child(int num)
9 {
10     //信号不排队，进入到wait回收，但并发50个信号，也只处理一个
11     //wait循环，则退出回调函数
12     //printf("child  %d is over\n",wait(NULL));

```

```

13     pid_t pid;
14     //循环回收，僵尸进程
15     while((pid = waitpid(0,NULL,WNOHANG)) > 0)
16         printf("child %d is over\n",pid);
17 }
18
19
20 void sys_err(char *str)
21 {
22     perror(str);
23     exit(1);
24 }
25
26
27 int main(void)
28 {
29     pid_t pid;
30     int m = 100;
31     while(m--)
32     {
33         if((pid = fork()) < 0)
34         {
35             sys_err("fork");
36         }
37         else if(pid == 0)
38         {
39             break;
40         }
41     }
42
43
44     if(pid == 0)
45     {
46         //子进程结束发送了SIGCHLD信号
47         int n = 5;
48         while(n--)
49         {
50             printf("child %d\n",getpid());
51             sleep(1);
52         }
53     }
54     else
55     {
56         //主进程

```

```

56 // 捕捉
57 struct sigaction act;
58 act.sa_handler = do_sig_child;
59 sigemptyset(&act.sa_mask);
60 act.sa_flags = 0;
61 sigaction(SIGCHLD,&act,NULL);
62
63
64 //父进程自己干自己的事
65 while(1)
66 {
67     printf("parent\n");
68     sleep(1);
69 }
70 }
71
72
73 return 0;
74 }

```

4.5、waitpid状态

```

1 pid_t waitpid(pid_t pid, int *status, int options);
2
3
4 options
5     WNOHANG    没有子进程结束，立即返回
6     WUNTRACED  如果子进程由于被停止产生的SIGCHLD， waitpid则立即返回
7     WCONTINUED  如果子进程由于被SIGCONT唤醒而产生的SIGCHLD， waitpid则立即返回
8 获取status
9     WIFEXITED(status)    子进程正常exit终止，返回真
10    WEXITSTATUS(status)   返回子进程正常退出值
11    WIFSIGNALED(status)   子进程被信号终止，返回真
12    WTERMSIG(status)      返回终止子进程的信号值
13    WIFSTOPPED(status)    子进程被停止，返回真
14    WSTOPSIG(status)      返回停止子进程的信号值
15    WIFCONTINUED(status)  子进程由停止态转为就绪态，返回真
16 见man 2 wait

```

5、向信号捕捉函数传参: sigqueue 函数（发送信号的同时携带数据）

```
1 int sigqueue(pid_t pid, int sig, const union sigval value);
2
3
4 union sigval
5 {
6     int sival_int;
7     void *sival_ptr;
8 };
```

有血缘关系，传地址才有意义

进程自己收发信号，在同一地址空间，父子进程

不同进程间收发信号，不在同一地址空间，不适合传地址

6、sigaction 捕捉函数 sa_flags = SA_SIGINFO 传参

```
1 void (*sa_sigaction)(int, siginfo_t *, void *);
2
3
4 siginfo_t
5 {
6     int si_int; /* POSIX.1b signal */
7     void *si_ptr; /* POSIX.1b signal */
8     sigval_t si_value; /* Signal value */
9     ...
10 };
11 sa_flags = SA_SIGINFO
```

7、信号中断系统调用

read阻塞时，信号中断系统调用后的两种情况：

- 1、 返回部分读到的数据（请求1000，但只读到100）
- 2、 read调用失败，error设成EINTR

进程间关系

1、 终端：是shell进程的控制终端，控制终端是保存在进程PCB中的信息。

```
xingwenpeng@ubuntu:~$ ps ajx
```

PPID	PID	PGID	SID	TTY	TPGID	STAT	UID	TIME	COMMAND
0	1	1	1	?	-1	Ss	0	0:19	/sbin/init
0	2	0	0	?	-1	S	0	0:00	[kthreadd]
2	3	0	0	?	-1	S	0	0:33	[ksoftirqd/0]
2	5	0	0	?	-1	S<	0	0:00	[kworker/0:0H]
2	7	0	0	?	-1	S	0	1:20	[rcu_sched]
2	8	0	0	?	-1	S	0	3:36	[rcuos/0]
2	9	0	0	?	-1	S	0	0:00	[rcuos/1]
2	10	0	0	?	-1	S	0	0:00	[rcuos/2]
2	11	0	0	?	-1	S	0	0:00	[rcuos/3]
2	12	0	0	?	-1	S	0	0:00	[rcuos/4]
2	13	0	0	?	-1	S	0	0:00	[rcuos/5]

带 ? 问号的是没有控制终端。有控制终端的进程，当控制终端结束后，进程也随之结束。

启动：init=>fork=>exec=>getty=>用户输入账号=>login=>输入密码=>exec=>shell

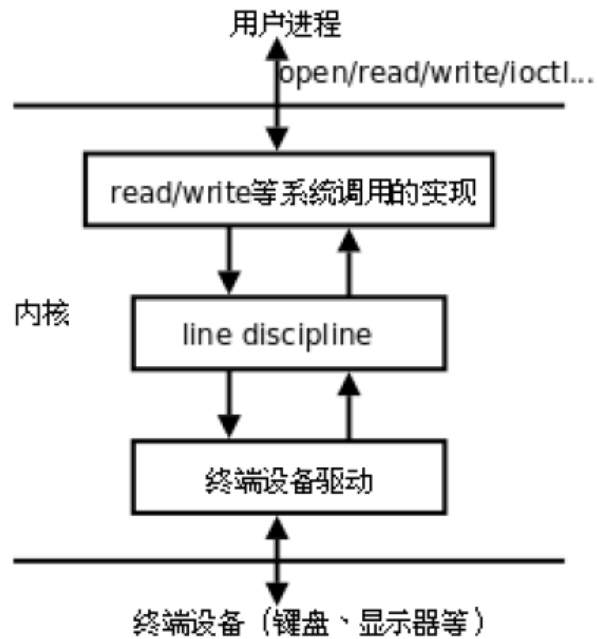
```
1 #include <stdio.h>
2 #include <unistd.h>
3
4
5 int main()
6 {
7     printf("fd 0:%s\n",ttyname(0)); //获取终端名字
8     return 0;
9 }
```

每个进程都可以通过一个特殊的设备文件/dev/tty访问他的控制终端。

ttyname函数可以查出，文件描述符的控制终端。

终端设备名称：用设备名/dev/tty表示，设备名从/dev/tty开始，依次递增，直到/dev/tty99为止。

终端的解析：用键盘输入信息传送到终端设备驱动，由终端设备给linux内核线程规程（过滤器）给read/write等系统调用实现。

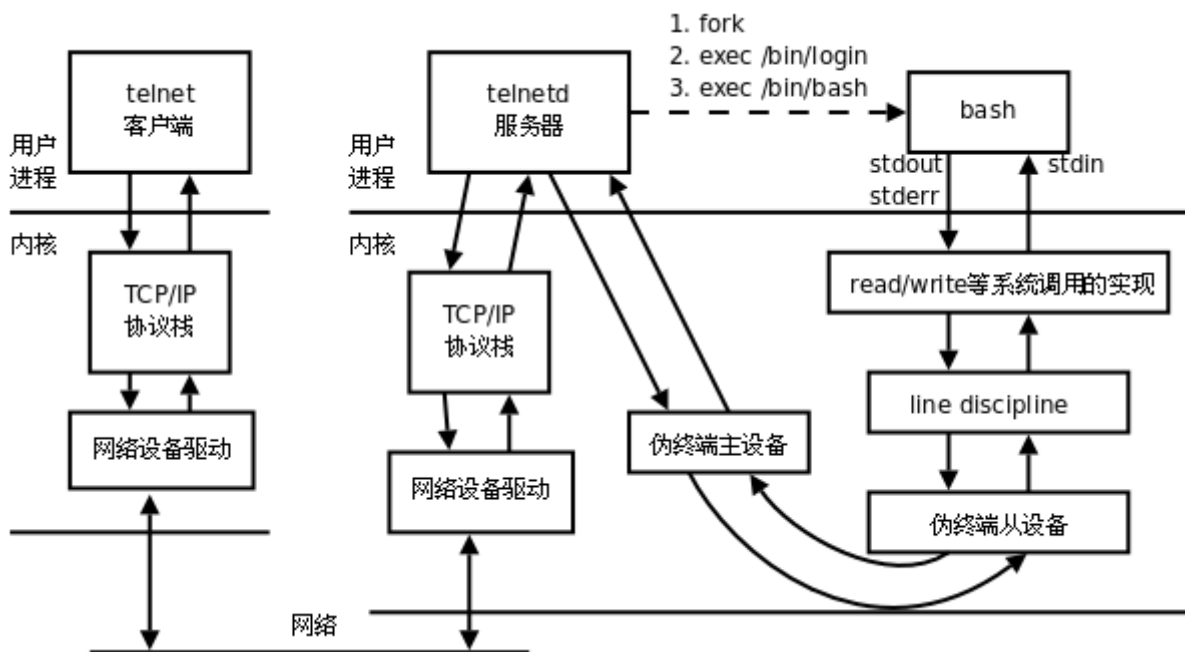


网络终端：有伪终端设备传送给linux内核规程（tcp/ip协议栈）

虚拟终端或串口终端的数目是有限，虚拟终端(字符控制终端)一般就是/dev/tty1

- dev/tty6六个，串口终端的数目也不超过串口的数目。然而网络终端或图形终端窗口的数目却是不受限制的，这是通过伪终端（Pseudo TTY）实现的。一套伪终端由一个主设备（PTY Master）和一个从设备（PTY Slave）组成。

每按一个键都要在网络上走个来回！



2、进程组

1)一个或多个进程的集合，进程组id是一个正整数，是由第一的进程号为组id号。

```
1 pid_t getpgid(pid_t pid); //获取pid进程的组ID
2
3
4 pid_t getpgrp(void); //获取当前进程的组ID
```

2)获取父子进程的组ID:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5
6 int main(void)
7 {
8     pid_t pid;
9
10
11     if ((pid = fork()) < 0)
12     {
13         perror("fork");
14         exit(1);
15     }
16     else if (pid == 0)
17     {
18         printf("child process PID is %d\n", getpid());
19         printf("Group ID is %d\n", getpgrp());
20         printf("Group ID is %d\n", getpgid(0));
21         printf("Group ID is %d\n", getpgid(getpid()));
22         exit(0);
23     }
24     sleep(3);
25     printf("parent process PID is %d\n", getpid());
26     printf("Group ID is %d\n", getpgrp());
27
28
29     return 0;
30 }
```

3)、组长进程标示:其进程组ID == 其进程ID

a)组长进程可以创建一个进程组，创建改进程组中的进程，被终止后(组ID不变)，只要进程组中有一个进程存在，进程组就存在，与组长进程是否终止无关。

b)进程组生存期：进程组创建到最后一个进程离开（终止或转移到另一个进程组）

c)一个进程可以为自己或子进程设置进程组ID (注意权限)

```
1 int setpgid(pid_t pid,pid_t pgid);
2
3
4 1)、如改变子进程为新的组，应在fork后，exec前使用；
5 2)、非root进程只能改变自己创建的子进程，或有权限操作的进程。
```

setpgid()加入一个现有的进程组或创建一个新进程组，如改变父子进程为新的组。

4)、修改组ID代码

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4
5
6 int main(void)
7 {
8     pid_t pid;
9     pid = fork();
10
11
12     if(pid == 0)
13     {
14         printf("child id %d,group id %d\n",getpid(),getpgid(0));
15         sleep(5);
16         printf("child id %d,group id %d\n",getpid(),getpgid(0));
17         exit(0);
18     }
```

```

19     else if(pid > 0)
20     {
21         printf("parent id %d,group id %d\n",getpid(),getpgid(0));
22         sleep(1);
23         setpgid(pid,pid);
24
25
26         printf("parent id %d,group id %d\n",getpid(),getpgid(0));
27         sleep(5);
28
29
30     }
31     else
32     {
33         return -1;
34     }
35
36
37     return 0;
38 }

```

```

xingwenpeng@ubuntu:~/LinuxCode/SignalIPC-07/kt$ ./a.out
parent id 3258,group id 3258
child id 3259,group id 3258
parent id 3258,group id 3258
child is 3259,group id 3259

```

3、 会话

```

1 pid_t setsid(void); //函数设置一个新的会话

```

- 1、 调用进程不能是进程组组长，该进程变成新会话首进程
- 2、 该进程成为一个新进程组的组长进程
- 3、 需要root权限（ubuntu不需要）
- 4、 新会话丢弃原有控制终端，该会话没有控制终端
- 5、 该调用进程是组长进程，则出错返回

6、建立新会话时，先调用**fork**，父进程终止，子进程调用**setsid**

```
1 pid_t getsid(pid_t pid);
2 //pid = 0表示查看当前进程session ID
```

7、**组长进程不能成为新会话首进程，新会话首进程必定成为组长进程。**

8、代码：

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5
6 int main(void)
7 {
8     pid_t pid;
9     pid = fork();
10    if(pid > 0)
11    {
12        while(1)
13            ;
14        exit(0);
15    }
16    else if(pid == 0)
17    {
18        setsid();
19        while(1)
20        {
21            ;
22        }
23    }
24    return 0;
25 }
```

1、**Daemon(精灵)进程**，是linux中的后台服务进程，生存期较长的进程，通常独立于控制终端并且周期性地执行某种任务或等待处理某些发生的事件。

2、守护进程的模型

- 1、创建子进程，父进程退出；所有工作在子进程中进行，形式上脱离了控制终端。
- 2、在子进程中创建新会话，**setsid()**函数，使子进程完全独立出来，脱离控制。
- 3、改变当前目录为根目录，**chdir()**函数，防止占用可卸载的文件系统，也可以换成其它路径。
- 4、重设文件权限掩码，**umask()**函数，防止继承的文件创建屏蔽字拒绝某些权限，增加守护进程灵活性。
- 5、关闭文件描述符，继承的打开文件不会用到，浪费系统资源，无法卸载。
- 6、开始执行守护进程核心工作。
- 7、守护进程退出处理。

3、代码：

```
1 #include <stdio.h>
2 #include <sys/stat.h>
3 #include <sys/types.h>
4 #include <fcntl.h>
5 #include <stdlib.h>
6 #include <unistd.h>
7
8
9 void daemonexe(void)
10 {
11     pid_t pid;
12     if((pid = fork())< 0)
13     {
14         perror("fork");
```

```
15     exit(1);
16 }
17 else if(pid > 0)
18 {
19     exit(0);
20 }
21 else
22 {
23     setsid();//创建一个新会话
24     if(chdir("/") < 0)//修改运行的目录
25     {
26         perror("chdir");
27         exit(0);
28     }
29     umask(0);//修改权限掩码
30     close(0);//关闭文件描述符
31     open("dev/null",O_RDWR);
32
33
34     dup2(0,1);//重定向文件描述符
35     dup2(0,2);
36
37
38     return;
39 }
40 }
41
42
43 int main(void)
44 {
45     daemonexe();
46     while(1)
47     {
48         ;
49     }
50     return 0;
51 }
```

4、时间记录的守护进程

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/stat.h>
6 #include <fcntl.h>
7 #include <string.h>
8 #include <time.h>
9
10
11 void sys_err(const char *str)
12 {
13     perror(str);
14     exit(-1);
15 }
16
17
18 void daemon()
19 {
20     pid_t pid;
21
22
23     if((pid = fork()) < 0)
24     {
25         sys_err("fork");
26     }
27     else if(pid > 0)
28     {
29         exit(0);
30     }
31     setsid();
32     if(chdir("/") < 0)
33         sys_err("chdir");
34     close(0);
35     open("/dev/null", O_RDWR);
36     dup2(0, 1);
37     dup2(0, 2);
38 }
39
40
41 int main(void)
42 {
43     daemon();
44     int fd.len:
```

```
45     time_t tm;
46     char buf[1024];
47     fd = open("/time.log", O_CREAT | O_WRONLY | O_APPEND, 0644);
48     while(1)
49     {
50         memset(buf, 0, sizeof(buf));
51         time(&tm);
52         localtime(&tm);
53         ctime_r(&tm, buf);
54         len = strlen(buf);
55         buf[len] = '\\0';
56         write(fd, buf, len);
57         sleep(5);
58     }
59     return 0;
60 }
```